Decentralized Storage Pooling: A Distributed Architecture for Fault-Tolerant and Secure Collaborative Storage Systems

Minor Project Report

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & TECHNOLOGY

by

Name	**Roll No.**
Dhakshinesh Anandh	500119339
Janhvi Shukla	500123433
Anuja Kotnala	500121914

Under the guidance of

Dr. J.Dhiviya Rose

School of Computer Science, UPES

Bidholi, Via Prem Nagar, Dehradun, Uttarakhand

May – 2026

CANDIDATE'S DECLARATION

We hereby certify that the project work entitled **"Decentralized Storage Pooling"** in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF TECHNOLOGY** in COMPUTER SCIENCE AND ENGINEERING with specialization in Full Stack Development and submitted to the Cloud Security and Operations Cluster, School of Computer Science, UPES, Dehradun, is an authentic record of my/our work carried out during a period from **Jan, 2026** to **May, 2026** under the supervision of **J.Dhiviya Rose, AI Cluster, UPES**.

The matter presented in this project has not been submitted by me/us for the award of any other degree of this or any other University.

Dhakshinesh Anandh	**Janhvi Shukla**	**Anuja Kotnala**
SAP Id. 500119339	SAP Id. 500123433	SAP Id. 500121914

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: _____ 2026 Project Guide

Acknowledgement

We wish to express our deep gratitude to our guide **J.Dhiviya Rose**, for all advice, encouragement and constant support she has given us throughout our project work. This work would not have been possible without his support and valuable suggestions.

We sincerely thanks **Dr.Christalin Nelson** our respected Cluster Head, Cloud Security and Operations, School of Computer Science, for his great support in doing our project Decentralized Storage Pooling .

We are also grateful to Dean SoCS UPES for giving us the necessary facilities to carry out our project work successfully. We also thanks to our Course Coordinator, **Dr.Abhishek Roshan** and our Activity Coordinator, **Dr.Deepa Rani** for providing timely support and information during the completion of this project.

We would like to thank all our friends for their help and constructive criticism during our project work. Finally, we have no words to express our sincere gratitude to our parents who have shown us this world and for every support they have given us.

Dhakshinesh Anandh	**Janhvi Shukla**	**Anuja Kotnala**
SAP Id. 500119339	SAP Id. 500123433	SAP Id. 500121914

Abstract

The recent exponential increase in digital data has created a strong need for storage systems which need to meet three requirements. The traditional cloud storage system which relies on centralized control has become the most common storage solution for organizations yet it contains multiple fundamental constraints. The system operates with single points of failure which lead to high operational and subscription expenses and it also restricts user data control while exposing customers to higher risks of cyber attacks and service interruptions. The challenges of the current system demonstrate the urgent need for new storage systems which can operate without needing centralized systems yet still provide dependable service and protection against data breaches.

The Distributed Storage Pooling System (DSPS) operates as a decentralized storage framework which uses unused storage space from common computing devices to create a unified storage network. The system uses local storage from users which allows the system to distribute content across different independent storage nodes. The system creates multiple storage options to enhance content accessibility while removing dependence on one specific storage system.

The system processes uploaded files through an adaptive chunking mechanism which divides files into chunks of different sizes according to their actual size and the current operational state of the system. The system achieves improved storage space utilization through its data transfer optimization techniques which also enhance its ability to recover from failures. The system uses robust encryption methods to protect each data chunk which ensures that data remains private during the entire process even when security breaches occur at storage nodes.

The system implements redundancy mechanisms which use RAID technology to provide both fault tolerance and continuous system operation. The system creates and distributes redundant chunks to multiple nodes which enables data reconstruction when nodes fail networks become unavailable or data gets corrupted. The metadata management layer stores fundamental details which include chunk locations and encryption mappings and redundancy configurations that support fast file access and restoration processes.

The system uses operating-system-level mutual exclusion to implement synchronization and concurrency control which maintains system integrity during simultaneous file access and modification by multiple users. The system prevents race conditions which helps maintain system reliability throughout multiple user interactions.

The proposed DSPS system delivers a secure and scalable solution which costs less than traditional cloud storage systems while achieving better storage efficiency and protecting data security and giving users data control and maintaining system reliability.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 History

Data storage systems have evolved from their original method of using centralized local storage to current solutions which employ extensive cloud-based storage systems. The original method of data storage used to save information on personal computers through hard drives and local servers that restricted users from accessing data and prevented them from expanding their systems. The development of internet technology enabled the creation of centralized cloud storage platforms which include Google Drive and Dropbox and Amazon S3 to provide users with remote data storage and access solutions.

The system of centralized control which operated as the main storage method for databases created multiple issues because it required users to pay for services they could not manage while their data remained vulnerable to outages which affected the whole system. Central server operations shut down all services during any central server downtime.

People started looking into decentralized storage systems as a solution to these problems. Peer-to-peer (P2P) networks together with distributed storage systems such as Tahoe-LAFS enabled users to store information across different independent storage nodes. The systems delivered improved data availability together with fault tolerance capabilities although their design failed to provide effective chunking processes and strong encryption systems as well as synchronized operational functions.

The proposed Distributed Storage Pooling System (DSPS) builds upon these concepts by combining adaptive chunking, encryption, redundancy, and synchronization into a unified decentralized framework.

1.2 Requirement Analysis

The project needs to examine its requirements because current centralized storage systems show their operational shortcomings and the project synopsis describes essential system functions.

Functional Requirements

- The system must enable users to upload and store files while providing them with efficient retrieval capabilities.

- The system needs to create file segments through its implementation of adaptive chunking technology.
- The system needs to encrypt all data chunks before they proceed to the storage process.
- The system needs to spread data chunks throughout various contributor nodes.
- The system needs to produce extra data chunks to establish fault tolerance which requires multiple backup chunks.
- The system needs to use existing metadata together with available data chunks for file reconstruction.
- The system uses synchronization mechanisms to control multiple users who access the system simultaneously.

Non-Functional Requirements

- Security The system must use encryption techniques to protect data confidentiality.
- The system requires uninterrupted operation during node outages according to its reliability requirement.
- The system needs to handle more users and additional storage nodes in the future.
- The system achieves better upload and download speeds through its ability to transfer multiple chunks at once.
- The system enables recovery through the use of backup chunks.

1.3 Main Objective

The main goal of this project is to build a decentralized storage system that aggregates the unused storage space of different devices into a common storage pool. The system creates a secure data storage solution which operates efficiently and withstands system failures without depending on centralized cloud services.

1.4 Sub Objectives

- The system needs adaptive chunking technology to achieve its purpose of effective file segmentation.
- The system requires encryption methods to protect data storage through secure data storage.
- The system needs to create redundancy systems which follow RAID design principles for its operations.
- The team will create a distributed storage system which operates across several storage devices.

- The system needs to create a metadata system which will help keep track of information and assist in its recovery process.
- The system uses mutual exclusion to maintain both synchronization and synchronization.
- The system needs to achieve two goals through its current process. The first goal is to maximize storage efficiency while the second goal is to decrease its reliance on cloud services.

1.5 System Workflow

The system uses a defined process to handle file storage and retrieval operations in its decentralized system.

- File Upload: The first step requires the user to upload a file which will be processed by the system.
- Chunking: The file undergoes adaptive chunking which creates smaller segments of the original file.
- Encryption: The system encrypts each chunk of data through its separate encryption process.
- Distribution: The system distributes encrypted data chunks to different nodes for storage purposes.
- Storage: The system stores data chunks on the devices which belong to contributors.
- Retrieval: The system first collects the necessary chunks before performing decryption and reconstruction to obtain the complete file.

Chapter 2

System Analysis

2.1 Introduction

The rapid growth of digital data has led to the development of various storage systems which improve scalability and security and reliability of stored data. Users prefer centralized storage systems because these systems provide simple operations and easy accessibility to their stored data. The system has three main drawbacks which include its single point of failure and its need for service providers and its expensive operational requirements.

The system faces these challenges but decentralized storage systems provide a solution through their different approach. These systems distribute data across multiple nodes which leads to better fault tolerance and increased accessibility to stored information. Researchers have tested different methods which include encryption and redundancy and peer-to-peer communication and distributed file systems to improve system performance and security.

This chapter examines current decentralized storage solutions while documenting their advantages and disadvantages according to the proposed system.

2.2 Existing System

The development of multiple decentralized storage solutions serves to solve the problems which arise from using centralized storage systems.

- The decentralized cloud storage system developed by [1]Srikanth et al. uses personal computer unused storage space to create distributed storage systems that optimize resource use.
- [2]Bacis et al. developed a decentralized storage system which uses AES encryption together with cryptographic access control functions to provide secure data protection.
- [3]Narendra et al. studied decentralized storage systems for IoT environments which needed to handle large scale operations while providing effective ways to share data.
- The decentralized storage model developed by [4]Khayrutdinov et al. demonstrates how systems can expand their capacity while maintaining effective storage operations.

- The research conducted by [5]Li et al. examined decentralized storage networks through an examination of different network protocols and security protection methods which exist in those networks.
- [6]Danezis et al. developed the Walrus system which combines encryption technology with erasure coding to provide users with an efficient method for restoring lost data.
- [7]Zhang et al. established a decentralized storage network which enables Private Information Retrieval (PIR) to provide users with better data security.
- [8]Guo et al. created a Byzantine Fault-Tolerant decentralized storage network which maintains system functionality when malicious nodes enter the network.
- [9]Selimi and Freitag developed Tahoe-LAFS to provide distributed storage through a system which protects data accessibility by implementing redundancy processes.

  The research shows that decentralized systems require an integrated approach which combines security measures with fault tolerance and distributed storage solutions.

2.3 Comparative Analysis

2.4 Research Gap

The literature survey shows that decentralized storage systems which currently exist focus on specific technical elements which include encryption and redundancy and scalability. The systems available in the market today do not deliver a comprehensive solution which combines all necessary system elements in a functioning manner.

Many approaches either lack strong encryption mechanisms, efficient fault tolerance strategies, or proper synchronization for concurrent access. Theoretical systems and practical systems exist in multiple systems which have not achieved real-world operational capabilities.

The proposed Distributed Storage Pooling System addresses these gaps through its implementation of adaptive chunking and encryption and redundancy and synchronization which create a complete framework that provides security and operational efficiency and system dependability.

Table 2.1: Comparative Analysis of Decentralized Storage Systems

Citation	Fault Tolerance	Encryption	Remarks
[1] Srikanth et al.	Not Applied	Applied (Basic)	Uses unused PC storage; focuses on decentralized resource utilization
[2] Bacis et al.	Not Applied	Applied	Strong cryptographic access control for decentralized cloud storage
[3] Narendra et al.	Not Applied	Discussed	IoT-oriented decentralized cloud storage architecture
[4] Khayrutdinov et al.	Not Applied	Discussed	Decentralized storage model with implementation perspective
[5] Li et al.	Discussed	Discussed	Systematic study of DSN protocols, proofs, and security mechanisms
[6] Danezis et al.	Applied	Partially Applied	Walrus system with erasure coding and self-healing recovery
[7] Zhang et al.	Partially Applied	Applied	Privacy-preserving retrieval using PIR integration
[8] Guo et al.	Applied	Applied	Byzantine Fault-Tolerant decentralized storage network
[9] Selimi and Freitag	Applied	Applied	Tahoe-LAFS distributed storage with redundancy mechanisms

Chapter 3

System Design

3.1 Proposed System

The system named Distributed Storage Pooling System (DSPS) creates a decentralized storage system which uses unused storage from multiple computing devices to form a unified storage pool. The system first divides data into multiple segments which it sends to various nodes because this method protects against system breakdowns while improving system performance.

Users can share some of their private storage space which the system transforms into accessible network storage. The system breaks uploaded files into smaller sections through its use of adaptive chunking techniques. The system protects data through encryption of parts which it distributes to different network nodes.

The system creates extra data copies which it saves on several nodes by using RAID-based systems to enhance fault protection. The metadata management system stores information about chunk locations and encryption details and redundancy mappings which enable fast file reconstruction.

The system implements synchronization techniques that use mutual exclusion to ensure data integrity during simultaneous access by multiple users. The proposed system establishes a secure storage solution which supports efficient growth while protecting against system failures, which serves as an alternative to existing centralized cloud storage solutions.

3.2 System Architecture

The system implements its modular construction and layered architectural framework through various components which work together to provide effective decentralized storage.

The system consists of main components which include:

- **Client Layer:** The user interface enables users to upload and download files while they manage their file-related tasks.
- **Chunking Engine:** The system uses chunking technology to create smaller file segments which it develops through its chunking operations.
- **Encryption Module:** The system creates encrypted copies of all data segments to ensure data confidentiality.

- **Redundancy Module:** The system achieves fault tolerance through its implementation of additional redundant components which create additional redundant data blocks.
- **Synchronization Manager:** The system uses mutual exclusion mechanisms to preserve system operational consistency.
- **Metadata Manager:** The system monitors location information about chunks and details regarding the file structure.
- **Storage Nodes:** The system operates as storage devices which store encrypted data chunks.

The system architecture permits independent operation of its components while it achieves system-level coordination through controlled metadata communication protocols.

3.3 Modules

The system consists of different modules that handle their designated tasks:

- User Interface Module: The module manages user interactions and handles authentication and file operations.
- File Management Module: The module handles both the upload and retrieval processes together with the file reconstruction tasks.
- Chunking Module: The system uses adaptive chunking to implement file division through its chunking module.
- Encryption Module: The module handles both the encryption and decryption processes for data chunks.
- Redundancy Module: The module creates backup chunks which enable the system to maintain fault tolerance.
- Synchronization Module: The module establishes mutual exclusion as the method for managing concurrent access to resources.
- Metadata Module: The module keeps track of all chunk locations together with the complete file structure of the system.
- Network Module: The module manages all communication activities that occur between different network nodes.

3.4 Design Diagrams

3.4.1 Use Case Diagram

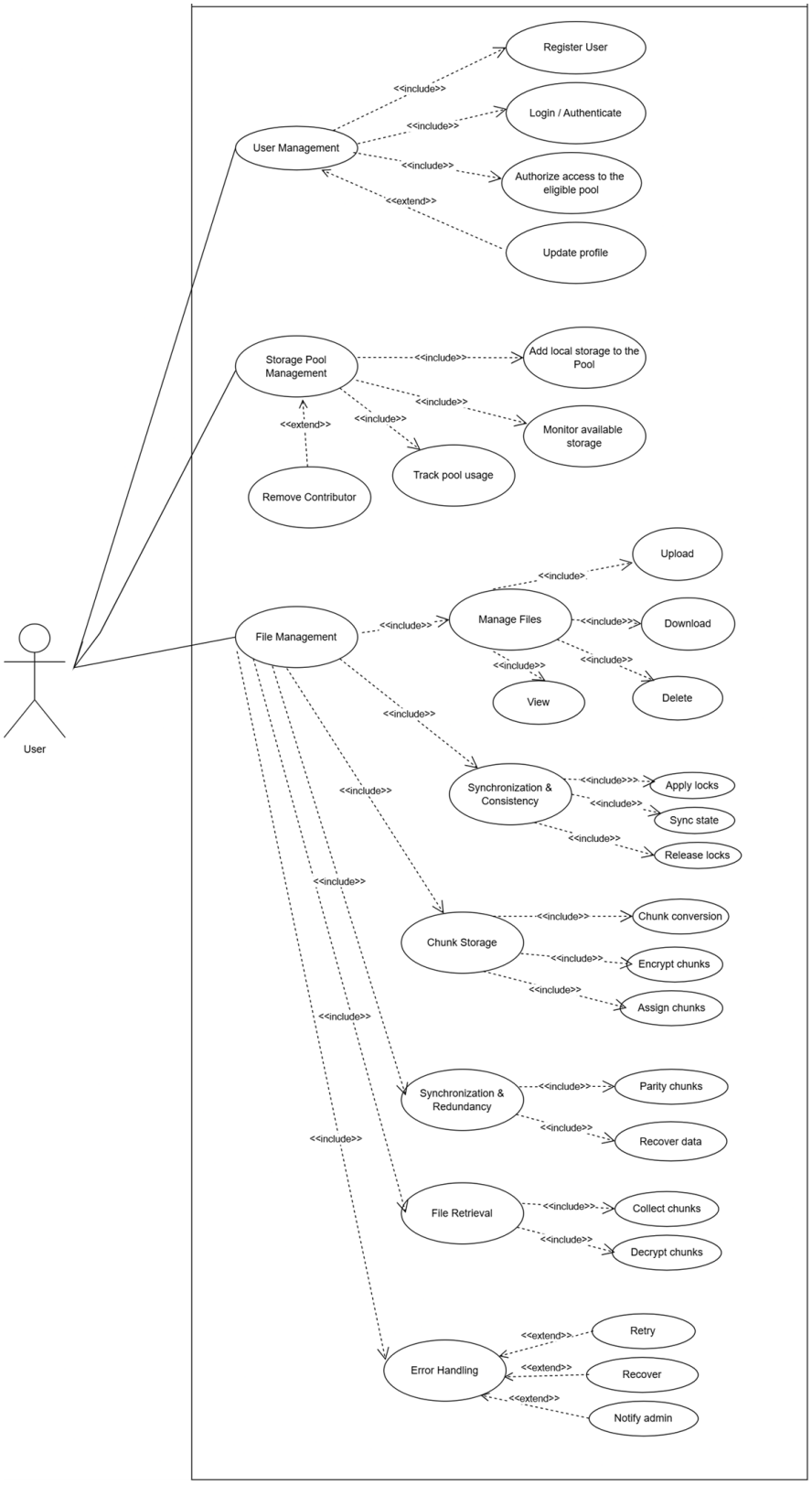

Figure 3.1: Use Case Diagram of DSPS

3.4.2 Activity Diagram

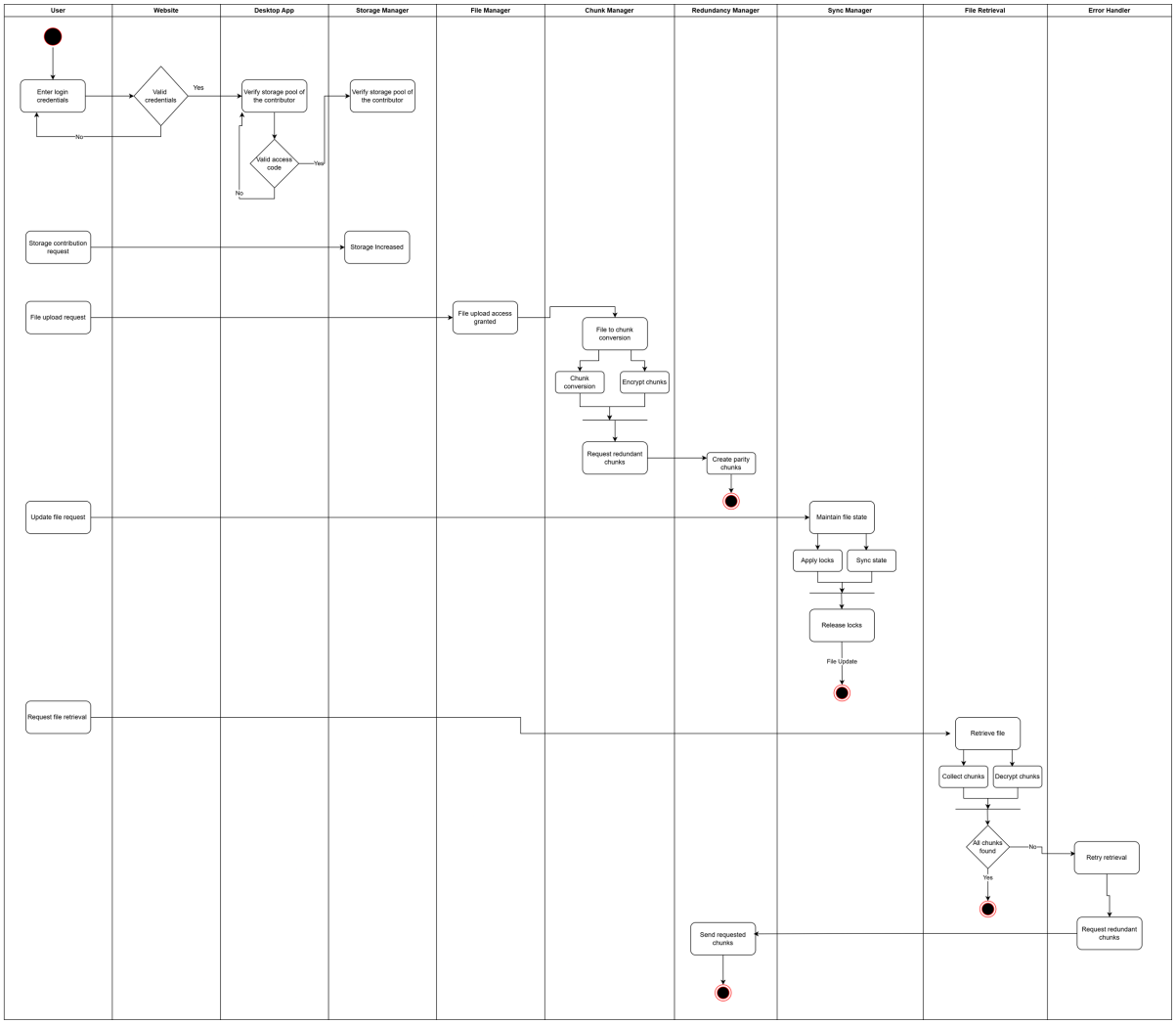

Figure 3.2: Activity Diagram of File Processing

Chapter 4

Technology Stack and System Specifications

4.1 Storage Technology Overview

The Decentralized Storage Pool (DSP) is implemented using a distributed storage system which operates through multiple nodes that provide their storage capacity to create an integrated storage network. The system achieves reliability and scalability through its implementation of chunk-based storage, secure communication methods, and redundancy mechanisms.

4.2 Core Storage Mechanisms

4.2.1 Technical Specifications

- Storage Model: Distributed peer-assisted architecture.
- Chunking Mechanism: Adaptive chunking, chunk size is decided by the number of nodes that are present in the DSP.
- Redundancy Mechanism: RAID 6-based parity which required a minimum of 4 nodes.
- Encryption Mechanism: AES-256 symmetric block cipher.
- Communication Protocol: REST APIs over HTTP.
- Node Participation: The system allows dynamic node participation which means nodes can join or leave pools any time.
- Metadata Management: Centralized tracking of metadata file which contains chunk information.

4.2.2 Key Features

- Distributed storage that does not depend on a central server for storage requirements.

- Efficient file handling through individual chunk control.
- Redundancy to provide falut tolerance.
- Secure data storage using encryption techniques.
- Dynamic addition and removal of nodes provide a scalable architecture.

4.2.3 System Architecture

The system follows a layered architecture consisting of:

- Client Layer: Web application and desktop applications.
- API Layer: Handles communication between the web application and the desktop application.
- Service Layer: Provides authentication, pool management, file operations and user dashboard.
- Storage Coordination Layer: Responsible for chunk allocation, redundancy, and node management.
- Data Layer: Stores chunk metadata and manages distributed storage nodes

4.3 Technology Stack

4.3.1 Software Components

- Programming Language: Python for desktop application
- Web Technologies: Angular for frontend
- Backend Communication: REST APIs
- Data Storage: Local file system for chunk storage

4.3.2 System Interfaces

- Desktop Application and Web Application - Interface for user interaction and node operations
- API endpoints for internal communication

4.4 Design Considerations

4.4.1 Scalability

New nodes can join the storage pool dynamically without affecting the existing operations which allows horizontal scaling of the system.

4.4.2 Reliability

RAID 6 allows failure of upto 2 nodes, which means data recovery is ensured and the system stays reliable.

4.4.3 Security

Encryption ensures data confidentiality while access control mechanisms safeguard the system against unauthorized access.

4.4.4 Resource Utilization

Chunk distributions is based on the storage contributed by a node which provides optimal resource utilization.

Chapter 5

Implementation

5.1 Environmental Setup

5.1.1 Users

A set of users who require external storage space and possibly having a common need for the data being stored in the decentralized storage pool(DSP). These users are required to have a PC with any operating system having an active internet connection without any interruption and possess required amount of storage space. RAID6 feature requires a minumum of 4 users connected to the DSP.

5.1.2 Software

A python based desktop application will be downloaded on each user's device. The users can have an access to the services by utilizing the terminal based UI or use the API endpoints on a website that is hosted on each user's device.

User Registration:

Each user is prompted with entering their username and directory path to store the shared chunks and downloaded files. The users will be uniquely identified by their sid.

Stroage Pool:

The users have the ability to create and join different storage pools. Once a user creates a storage pool, they will be given the access code and the user can share the access code among their peers. The peers can request to join the storage pool, and hence the pool owner can accept or decline the join request.

5.2 Decentralised Stroage Pool Phases

5.2.1 Adaptive Chunking

Upload:

When a file gets uploaded to the DSP, the file gets divided into chunks based on the file size optimising upload experience by sending smaller chunks so that during any failure,

the chunk can be sent again in lesser time. This saves time and enhances experience of upload.

A Metadata file will be created which contains the following information about the chunks:


```
{"file_id": "id",
"file_name": "video.webm",
"file_extension": ".webm",
"owner_user_id": userID,
"storage_pool_id": poolID,
"version": 1,
"upload_timestamp": "2026-03-12T19:27:36.813381+00:00",
"original_file_size": 18172711,
"chunk_size": 9086356,
"total_chunks": 2,
"last_chunk_size": 9086355,
"chunking_strategy": "Adaptive",
"chunks": [
  {
    "chunk_id": "id",
    "chunk_index": 0,
    "storage_node_id": id,
    "byte_offset": 0,
    "chunk_size": 9086356
  },
  {
    "chunk_id": "8963154b-995e-4eb2-9f14-7be24fc7c7c5",
    "chunk_index": 1,
    "storage_node_id": id,
    "byte_offset": 9086356,
    "chunk_size": 9086355
  }
],
}
```


Download:

The chunk will be retrieved from each peer and, once all the chunks are retrieved, the original file will be reconstucted and stored in the user's download directory.

5.2.2 Encryption

Upload:

Each chunk will be sent through encryption ensuring any person who owns these chunks can not obtain the chunk information without the key with which the chunks were encrypted.

Download:

The Chunks will be decrypted using the provided key once all the chunks will be retrieved from the peers.

The Sender will be provided with the key of the encryption after upload is successful. He/She can send the key to the peers using the message service if they want to provide access to the file data.

5.2.3 Redundancy(RAID6)

The DSP system requires a minimum of 4 users to enable the RAID6 feature. This feature helps in creating redundant chunks among peers. This creates an ability to retrieve the data even when two peers get disconnected from the DSP.

5.2.4 Synchronisation and Mutual Exclusion

This module ensures no two file/chunks are having write-write, read-write conflicts. These conflicts occur when two files with the same file name and it is prevented by creating two different file id's so that each user has their own data.

5.2.5 API

The application is enabled with API endpoints. The user can develop any software which can access the functionalities provided by the DSP.

5.3 Algorithms

5.3.1 AES-256 Encryption

The Decentralized Storage Pool (DSP) system employs the Advanced Encryption Standard (AES) with 256-bit keys to ensure confidentiality and security of user data.

AES-256 is a symmetric key encryption algorithm that operates on fixed-size blocks of 128 bits using a 256-bit encryption key. It transforms plaintext into ciphertext through multiple rounds of substitution, permutation, and mixing operations.

$$C = E_K(P), \quad P = D_K(C) \tag{5.1}$$

where:

- P = Plaintext (original chunk data)
- C = Ciphertext (encrypted chunk)
- K = 256-bit encryption key
- E_K = Encryption function
- D_K = Decryption function

Algorithm 1: File Upload in Decentralized Storage Pool

Input: File F, Storage Pool SP, User U
Output: Metadata M, Encryption Key K
Generate unique file ID fid;
Divide F into chunks $C_1, C_2, ..., C_n$ using adaptive chunking;
Generate 256-bit encryption key K;
for *each chunk* C_i **do**
    Encrypt chunk: $E_i = AES\_Encrypt(C_i, K)$;
if *RAID6 enabled* **then**
    Compute parity chunks P and Q;
    Distribute E_i, P, Q across storage nodes;
else
    Distribute encrypted chunks E_i across nodes;
Create metadata M containing:

- file ID, chunk info, node locations
- encryption type, chunk sizes

Store metadata M;
Return K to user U;

Algorithm 2: File Download in Decentralized Storage Pool

Input: File ID fid, Encryption Key K
Output: Reconstructed File F
Retrieve metadata M using fid;
Fetch all chunks (or available chunks) from storage nodes;
if *some chunks missing AND RAID6 enabled* **then**
    Reconstruct missing chunks using parity (P, Q);
else
    Proceed with available chunks;
for *each encrypted chunk* E_i **do**
    Decrypt chunk: $C_i = AES\_Decrypt(E_i, K)$;
Reassemble chunks $C_1, C_2, ..., C_n$ into file F;
Store F in user download directory;
return F;

Advantages

- Strong resistance to brute-force attacks
- Efficient for large-scale chunk-based systems
- Ensures data confidentiality in decentralized storage
- Industry-standard encryption algorithm

5.3.2 RAID 6 (Redundant Array of Independent Disks)

RAID 6 is implemented in the DSP system to provide fault tolerance and redundancy. It allows recovery of data even if two storage nodes fail simultaneously.

RAID 6 distributes both data and parity information across multiple nodes. It uses dual parity, commonly denoted as P and Q.

$$P = D_1 \oplus D_2 \oplus \cdots \oplus D_n \tag{5.2}$$

where:

- $D_1, D_2, \ldots, D_n$ = Data chunks
- P = XOR-based parity
- $\oplus$ = Bitwise XOR operation

A second parity (Q) is generated using advanced techniques such as Reed-Solomon encoding, enabling recovery from two simultaneous failures.

RAID 6 Structure in DSP

- Minimum of 4 storage nodes required
- Data and parity are distributed across nodes
- Two parity blocks (P and Q) are maintained
- Parity placement is rotated for load balancing

Upload Process

1. File is divided into chunks.
2. Data chunks are distributed across nodes.
3. Two parity chunks (P and Q) are computed.
4. Parity chunks are stored on separate nodes.

Recovery Process

- Single node failure: recovered using XOR parity
- Two node failure: recovered using dual parity (P and Q)
- Missing chunks are reconstructed dynamically

Advantages

- High fault tolerance (up to 2 node failures)
- Improved reliability in decentralized systems
- No single point of failure
- Efficient for distributed storage environments

Chapter 6

Output Screens and Results

Figure 6.1: Set username and path of download and shared storage.

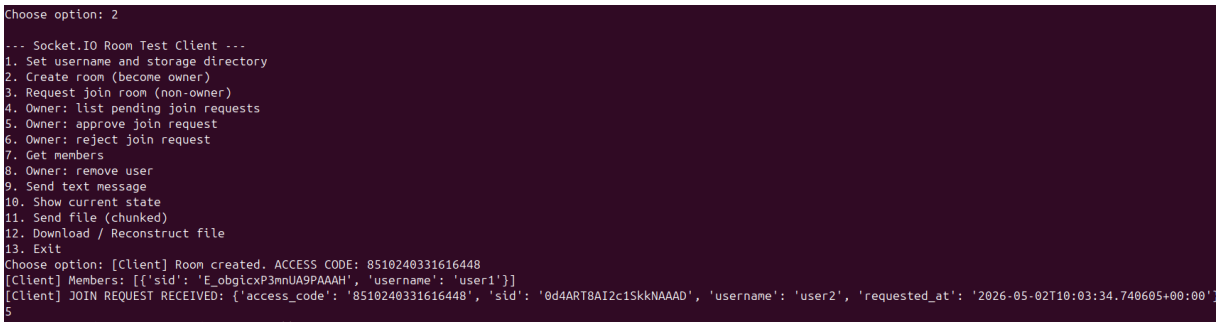

Figure 6.2: Create and accept join request to DSP using access code.

Figure 6.3: Upload the file to the DSP and obtain the encryption key.

Figure 6.4: Reconstruct and Download the file using the encryption key.

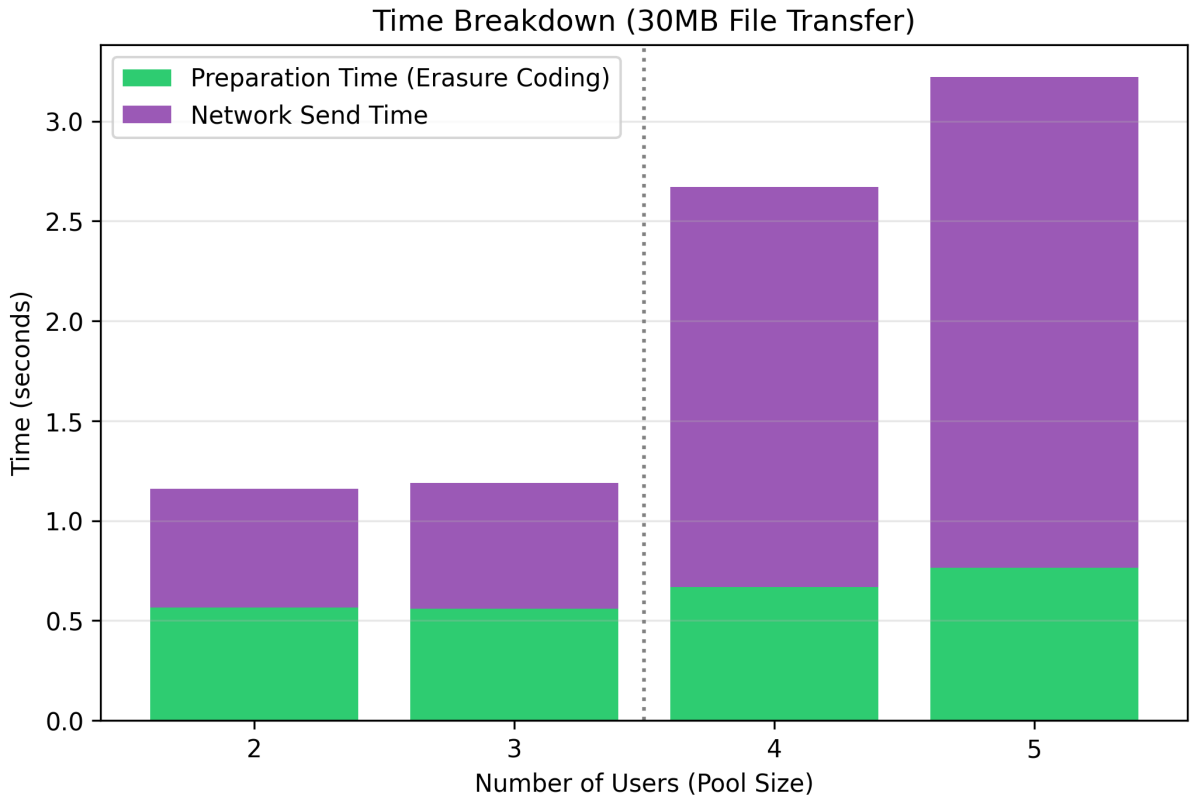

Figure 6.5: Time taken for each set of users to send a file in Round Robin and RAID6.

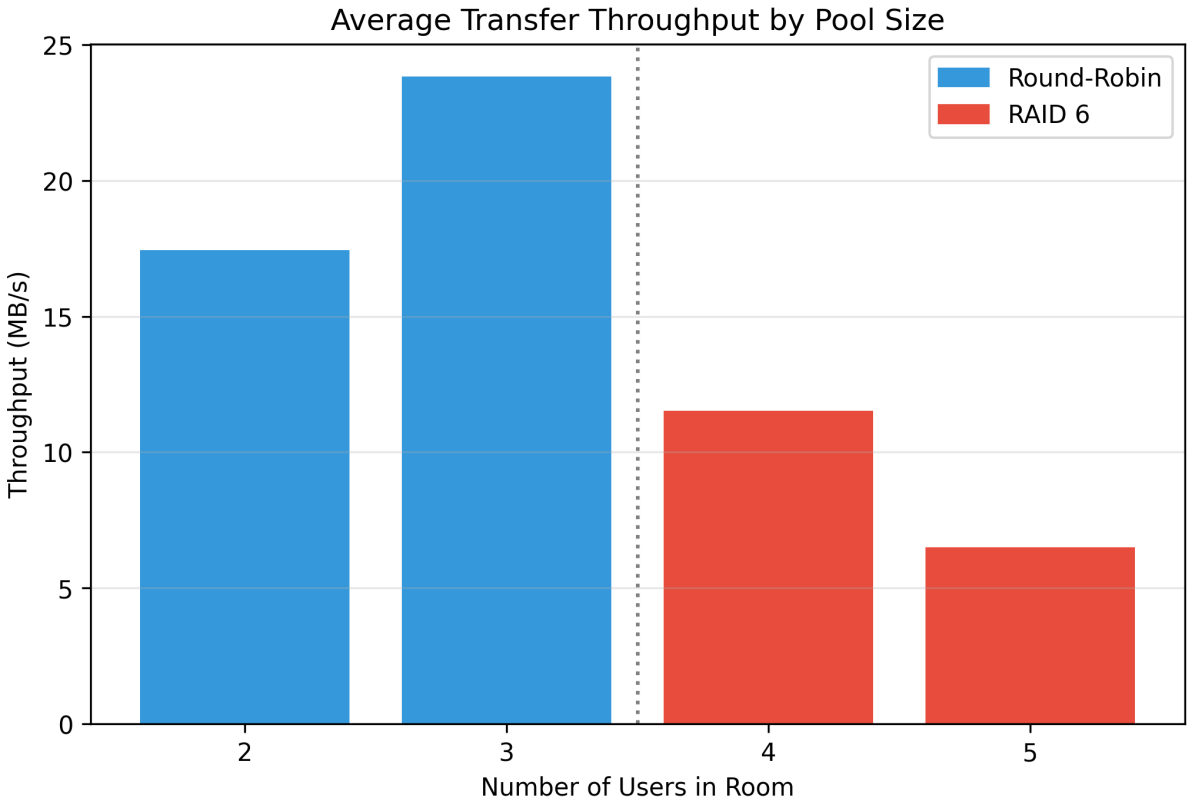

Figure 6.6: Throughput of each user in Round Robin and RAID6.

Chapter 7

Limitations and Future Enhancements

7.1 Limitations

- Dependence on active nodes: Availability of data depends on the number of online nodes in the storage pool.
- Network dependency: To ensure efficient and robust performance, a stable and high speed internet connection is required.
- Higher latency for contributors: Nodes that contribute less storage to the pool are allocated lesser number of chunks (adaptive chunking) and hence the latency for file reconstruction and download is higher for them.
- Limited fault tolerance: If more than 2 nodes fail at the same time, the system may not remain reliable.

7.2 Future Enhancements

7.2.1 Technical Improvements

- Priority based chunk allocation for nodes that require the shared files more frequently.
- Improved node selection algorithms based on performance, reliability, and network conditions.
- Improved fault detection and recovery techniques to improve system reliability.

7.2.2 Applications

The proposed system can be extended for use in:

- Genomic Data Storage: Genomic data introduces challenges in privacy, scale, and trust. Large files such as FASTQ, BAM, VCF require secure and reliable storage. Centralized systems struggle to scale while ensuring data protection, and blockchain-based systems are inefficient for large datasets due to high latency and storage limits.

A decentralized storage system offers an appropriate solution by distributing data across multiple nodes, allowing parallel access, better performance, while ensuring scalability. It can be deployed using institutional clusters, cloud environments, or private distributed networks, making it suitable for large-scale genomic data storage.

- Collaborative Research Data Storage: Research teams can use the system to store and share extensive datasets which include experimental results and simulations and media data. The system enables efficient collaborative work because it distributes data among multiple contributors while decreasing reliance on central storage systems and maintaining access to data during node failures. The method proves especially beneficial in academic and scientific settings which require dependable data sharing systems.

- Community-driven Cloud Storage Networks:The system functions as a storage system operation tool which enables users to contribute their unused storage space for the establishment of a shared storage infrastructure. The system provides an alternative to commercial cloud services while it reduces storage costs and enables users to share resources in a decentralized manner. The system uses data distribution across multiple participants to improve system availability while allowing the community to construct an expandable permanent storage solution.

Chapter 8

Conclusion

The Distributed Storage Pooling System (DSPS) developed in this project provides an effective storage solution, serving as an alternative to centralized cloud based storage. By combining adaptive chunking, AES-256 encryption, RAID 6 redundancy, and OS-level synchronization into an integrated decentralized framework, the system addresses the primary limitations of traditional storage solutions which include single points of failure, high subscription costs, limited user control, and vulnerability to data breaches.

The system successfully fulfills its core objectives. Files are securely chunked then encrypted and distributed across nodes, with the encryption key available exclusively to the uploader which reinforces data confidentiality. The RAID 6 technology provides robust fault tolerance, enabling data recovery from failure of upto 2 nodes concurrently, which makes the system reliable.

Performance evaluation showed a trade-off between Round Robin and RAID 6. Round Robin achieved higher throughput of 17 to 24 MB/s for 2 to 3 users while RAID 6 achieved lower throughput of 6 to 11 MB/s for 4 to 5 users. Better data protection through redundancy in RAID 6 makes this trade-off acceptable for decentralized environments.

The metadata management layer proved to be crucial for efficient file reconstruction because it stored chunk locations, encryption mappings and redundancy configurations. The mutual exclusion mechanism ensured system integrity during concurrent access by preventing race conditions across multiple users.

The DSPS project demonstrates that standard hardware and open technologies can be used to create a decentralized storage system which is secure and scalable and fault-tolerant. The project creates a solid foundation for future improvements including blockchain based access control systems and priority based algorithms for storage contributors along with dynamic node discovery and support for expanded distributed network systems which will open the doors for completely autonomous and user controlled storage ecosystems.

Appendix A

Keywords

Table A.1: List of Keywords and Their Definitions

Keyword	**Definition**
Decentralized Storage	A storage architecture where data is distributed across multiple independent nodes instead of a central server.
Distributed Systems	A system consisting of multiple interconnected computers that communicate and coordinate their actions to achieve a common goal.
Peer-to-Peer Networks	A network model in which each participant (node) can act as both a client and a server, sharing resources directly.
Adaptive Chunking	A technique that divides files into variable-sized chunks based on system conditions and file size for efficient storage and transfer.
AES-256 Encryption	A symmetric encryption algorithm using a 256-bit key to securely transform plaintext into ciphertext.
RAID 6	A redundancy mechanism that uses dual parity to allow data recovery even if two storage nodes fail simultaneously.
Fault Tolerance	The ability of a system to continue functioning properly even when some components fail.
Data Redundancy	The practice of storing duplicate copies of data to ensure reliability and recovery in case of failure.
Metadata Management	The process of storing and managing information about data, such as chunk locations and structure.
Mutual Exclusion	A synchronization mechanism that prevents multiple processes from accessing a shared resource simultaneously.
Synchronization	Coordination of multiple processes to ensure correct execution and data consistency.
Data Security	Techniques used to protect data from unauthorized access, corruption, or theft.
Cloud Storage Alternative	A system designed to replace centralized cloud storage with a decentralized or distributed approach.
Distributed File Systems	File systems that store and manage data across multiple machines while appearing as a single system to users.

References

[1] S. Srikanth, T. S. Kumar, S. Anamalamudi, and M. Enduri, "Decentralized cloud storage using unutilized storage in pc," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2021, pp. 1–5.

[2] E. Bacis, S. De Capitani di Vimercati, S. Foresti, S. Paraboschi, M. Rosa, and P. Samarati, "Securing resources in decentralized cloud storage," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 286–298, 2020.

[3] N. C. Narendra, K. Koorapati, and V. Ujja, "Towards cloud-based decentralized storage for internet of things data," in *2015 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, 2015, pp. 160–168.

[4] D. D. Khayrutdinov, D. A. Volkov, and A. G. Mukhina, "The model and application of data storage in a decentralized network," in *2024 IEEE 25th International Conference of Young Professionals in Electron Devices and Materials (EDM)*, 2024, pp. 1840–1843.

[5] C. Li, M. Xu, J. Zhang, H. Guo, and X. Cheng, "Sok: Decentralized storage network," *High-Confidence Computing*, vol. 4, no. 3, p. 100239, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667295224000424

[6] G. Danezis, G. Giuliari, E. K. Kogias, M. Legner, J.-P. Smith, A. Sonnino, and K. Wüst, "Walrus: An efficient decentralized storage network," 2026. [Online]. Available: https://arxiv.org/abs/2505.05370

[7] J. Zhang, M. Xu, H. Guo, and X. Cheng, "Pir-dsn: A decentralized storage network supporting private information retrieval," 2025. [Online]. Available: https://arxiv.org/abs/2512.07189

[8] H. Guo, M. Xu, J. Zhang, C. Liu, R. Ranjan, D. Yu, and X. Cheng, "Bft-dsn: A byzantine fault-tolerant decentralized storage network," *IEEE Transactions on Computers*, vol. 73, no. 5, p. 1300–1312, May 2024. [Online]. Available: http://dx.doi.org/10.1109/TC.2024.3365953

[9] M. Selimi and F. Freitag, "Tahoe-lafs distributed storage service in community network clouds," in *2014 IEEE Fourth International Conference on Big Data and Cloud Computing*, 2014, pp. 17–24.